Jerry Trader Documentation

JerryHong

Started December 2025
Last updated: March 11, 2026

Contents

1 Introduction **3**

2 Frontend Modules **5**
2.1 frontend overview 5
2.2 modules breakdown 6

3 Backend Architecture **6**
3.1 Package Layout 7
3.2 Computation Core (`core/`) 7
3.3 Services 7
3.4 Rust Migration Roadmap 8
3.5 Stage 2.5 — Bars Builder Pipeline 9
3.5.1 Core Design 9
3.5.2 Implementation Phases 10
3.5.3 10s Bootstrap — Historical Back-fill at Subscription Time 11
3.5.4 Multi-Timeframe Bootstrap — Extension Design Notes 13
3.5.5 UTC/ET Timezone Conversion — DST-Aware Session Boundaries 14

4 Roadmap **16**

1 Introduction

Summary Jerry Trader is a quant trading platform mainly focus on real-time US pre-market penny stock break out momentum trading strategy, written multi-language (Python + Rust + React). This documentation provides an overview of the app's features, architecture, and usage instructions *based on my trading strategies.*

The project follows a monorepo layout:

- `python` — Python backend (orchestration, I/O, backend services)
- `rust` — Rust computation core (compiled via maturin/PyO3 as `jerry_trader._rust`)
- `frontend` — React + TypeScript frontend (Vite)

Strategy Here is a summary of my strategy based on emperical trading experience, and the modules are designed to better validate, automate this strategy:

- Entry Attention Condition:
  when tickers are getting active, it has a amount of gains suddenly, we do the momentum trading. for my current project, I use Top20 Entry as a detector.
- It's Better If:
  - it has recent hot news with it.
  - the float shares are relatively low, below 20M is preferred.
  - the country location is better not foreign-registered, etc.
  - price is preferred $1–$20
  - the market has all the attention on this one.
  - Order Book liquidity good(intense trade rates with good up-rising momentum)
- It's Better NOT If:
  - Order Book liquidity good but with multiple key-level rejections.
  - price under 0.5
  - big cap companies earnings beat.

It's not saying that if all the conditions fit it will definitly be a good trade. the conditions will be better if all fit, but sometimes there are unknown black swarn ones, so the key metrics will still be price-volume action and orderbook analysis etc.and besides of that, for a long-term profitable trading strategy, also it depends your trading execution and risk management like:

- entry timing and price.
- risk management(marginal account risk etc).
- …

Feature for current version, here are some features that done:

- modules decoupled, driven by redis stream msg.
- rank list column and overview chart for top20 tickers visualization.
- instant news data fetch, process, LLM system prompt sentiment process, persistence.
- stock info data like: float data, basic information fetch, process, persistence.
- ibkr order execution,management,persistence to PostgreSQL.
- real-time bar build and persistence optimized using clickhouse&rust.
- live mode and replay mode support.
- rust based global clock for time accuracy.

2 Frontend Modules

2.1 frontend overview

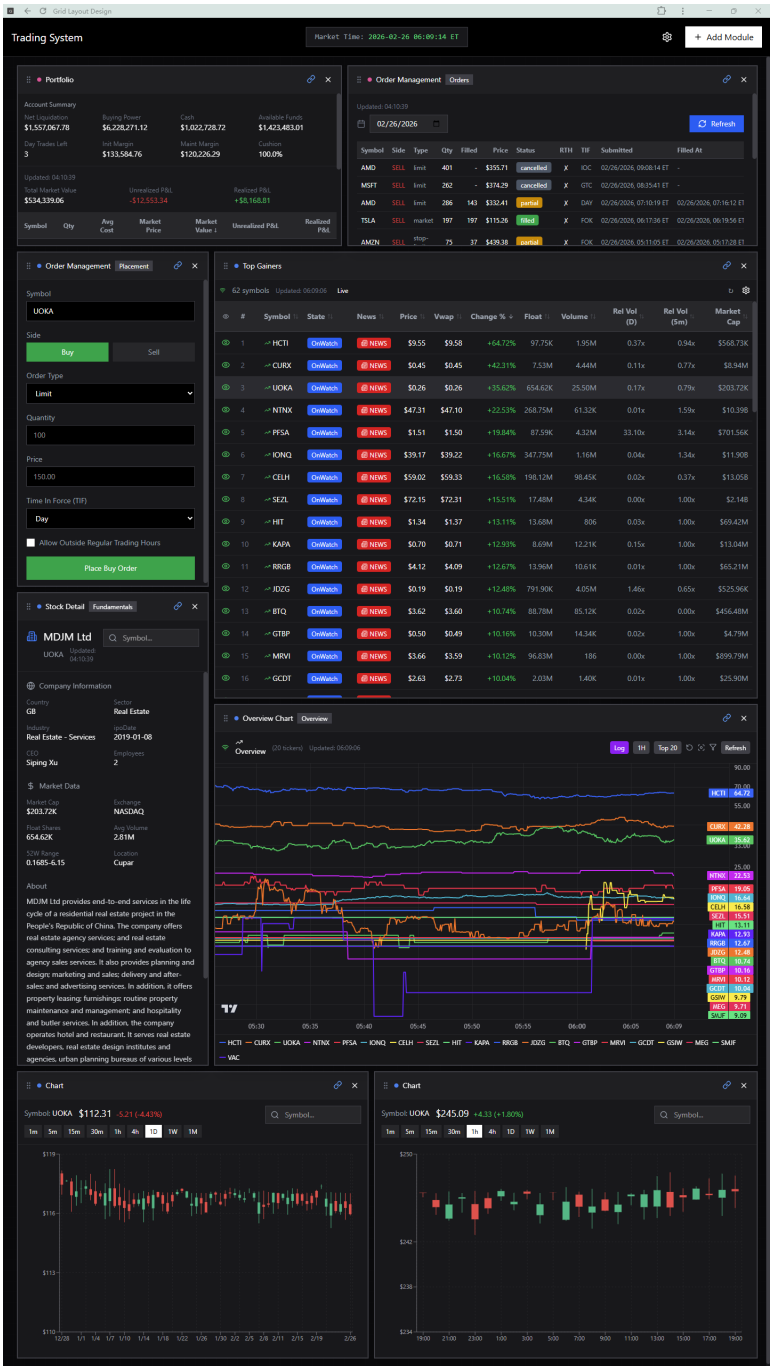

Figure 1: Jerry Trader Application Interface

Platform Overview So, as you can see above, this is my current version of frontend web app interface. but you might wonder, as a rigorous quant trading platform, why would I develop a trading platform that UI is designed for human-centered trade execution?
well, first, I just love trade, my goal is to be profitable quant trader, not researcher due to my lack of rigourous knowlegde of mathematics. second, this project is planned for long-term architecture, I don't have the experience and ability to develop a end-to-end tickdata to machine-decision execution quant platform. the graduate approach is the only choice.

2.2 modules breakdown

Module Type	**Sub-View Name**	**Description**	**Backend**
rank-list		rank list with key metrics	core.snapshot,static-data-worker,news-worker,core.signals
stock-detail	fundamentals	stock basic info	staticdata-worker
stock-detail	news		news-worker
overview-chart	overview		overviewchart-management
overview-chart	focus	focus on one tiker with state segmentation	overviewchart-management
chart			chart-data-service
portfolio		portfolio and positions	OrderManagement
order-management	placement		OrderManagement
order-management	orders		OrderManagement

Table 1: Module descriptions and their corresponding backends.

In the next Chapter, I will break down the details of each module in the above table, which will include:

- data preparation
- data process
- orchestration
- module design

3 Backend Architecture

not start to rewind yet. below is just lagacy llm-generated notes&summary

3.1 Package Layout

The Python backend is organized into the following top-level packages:

3.2 Computation Core (`core/`)

The `core/` package is the **Rust migration boundary**. Each subpackage follows a compute/orchestration split:

- `compute.py` — Pure functions & stateful trackers (**Rust migration target**). These have planned counterparts in `rust/src/`.
- `processor.py` / `engine.py` — I/O orchestration (Redis, InfluxDB, threads). Stays in Python.

The `_bridge.py` module provides a try/except dispatch:

```
try:
    from jerry_trader._rust import VolumeTracker,
    ↪  compute_ranks
except ImportError:
    from jerry_trader.core.snapshot.compute import
    ↪  VolumeTracker, compute_ranks
```

This allows incremental migration — Rust implementation when compiled, Python fallback otherwise.

3.3 Services

- **BackendForFrontend (BFF)**
  FastAPI + WebSocket server running on the main machine. Collects and preprocesses (`core.snapshot`); receives and emits rank data and overview-chart data to the frontend.
- **core.snapshot (SnapshotProcessor)**
  Receives raw market snapshots from Redis stream, computes ranks and derived metrics (via Rust when available), manages the top-movers subscription set, writes to output Redis stream and InfluxDB.
- **core.factors (FactorManager)**
  Real-time factor engine. Ingests tick-level trade data. Computes trade rate. Computes acceleration. Produces rolling z-scores. Outputs to InfluxDB. Uses Redis (HSET) for metadata. Includes historical bootstrap via Polygon API.
- **core.signals (StateEngine)**
  Polls factor HSETs and fires state transition events (QUIET → ACTIVE, ACTIVE → QUIET) based on z-score thresholds.
- **core.news (NewsProcessor)**
  LLM-based news classification. Determines if news articles are positive catalysts using DeepSeek API.

- **OrderManagement**
  Order management system based on Interactive Brokers API. Handles order placement, status callbacks, portfolio and position tracking. Backed by PostgreSQL via SQLAlchemy + Alembic.
- **DataSupply**
  External data sources:
  - `snapshotDataSupply` — market snapshot collection & replay
  - `tickDataSupply` — real-time tick data (Polygon.io, ThetaData, local Rust replay)
  - `staticdataSupply` — fundamentals, borrow fee, news fetch
  - `bootstrapdataSupply` — historical data loaders
- **DataManager**
  Data-serving workers that bridge DataSupply to the BFF:
  - `static_data_worker` — fetches fundamentals for subscribed tickers
  - `news_worker` — fetches and caches news
  - `tickdata_server` — serves tick data to frontend and factor engine
  - `chart_data_service` — historical bar data for TradingView chart
- **utils**
  Cross-cutting utilities: config, logger, Redis key naming, session management, path resolution.

3.4 Rust Migration Roadmap

The migration from Python `compute.py` modules to Rust follows a three-step progression, ordered from simplest to most complex:

1. **Step 1: Pure math functions** (`core/factors/compute.py`)
   `z_score` and `price_accel` — stateless, no external dependencies. This step establishes the full end-to-end workflow: write Rust in `lib.rs`, run `maturin develop`, `_bridge.py` auto-switches, existing tests validate correctness, `RUST_FACTORS` flips to `True`.

2. **Step 2: Stateful tracker** (`core/snapshot/compute.py`)
   `VolumeTracker` as a `#[pyclass]` with methods — learn stateful Rust objects exposed to Python via PyO3.

3. **Step 3: DataFrame operations** (`core/snapshot/compute.py`)
   `compute_ranks`, `compute_derived_metrics`, `compute_weighted_mid_price` — Polars DataFrame in/out, requires the `pyo3-polars` crate. Most complex.

Note: `core/signals/` (StateEngine) is excluded from the Rust migration roadmap. The signal/state detection logic depends on strategy rules that are not fully decided yet; it will remain Python-only for the foreseeable future.
Status: Step 1 completed. `z_score` achieved 3.6–10.7× speedup, `price_accel` achieved 8.9–12.1× speedup over Python. Steps 2–3 are deferred in favour of the Bars Builder pipeline below.

3.5 Stage 2.5 — Bars Builder Pipeline

The Chart module requires a reliable server-side bar-building pipeline. The v1.0 architecture (frontend fetches historical bars from Polygon API, real-time last-bar-only update from raw trade ticks in `chartDataStore.ts`) suffered from:

- Bar consistency issues across timeframes
- Duplicated aggregation logic (frontend *and* factor engine)
- No single source of truth for completed bars
- Cannot scale to feed downstream engines (factors, signals, risk)

The v2.0 architecture introduces a **Rust-native Bar Builder** as the canonical bar source for all downstream consumers:

```
Polygon / ThetaData / Replayer ticks
         |
         v
  UnifiedTickManager (Python, existing)
         |  asyncio.Queue
         v
  +----------------------------+
  |  BarBuilder (Rust #[pyclass])  |
  |  * ingest_trade()          |
  |  * get_current_bar()       |
  |  * completed-bar callbacks |
  +-------------+--------------+
                |
     +----------+----------+----------+
     |          |          |          |
     v          v          v          v
 ClickHouse  Frontend  FactorEngine  Future:
 (persist)   (WS push)  (bars)     signals, risk
```

3.5.1 Core Design

Timeframes. Eight concurrent timeframes per ticker: 10 s, 1 m, 5 m, 15 m, 1 h, 4 h, 1 d, 1 w. Each is maintained as an independent rolling `BarState` inside the Rust `BarBuilder`.

Session awareness. Bars respect US market session boundaries:

- Premarket: 04:00–09:30 ET
- Regular: 09:30–16:00 ET
- Afterhours: 16:00–20:00 ET

A bar never spans a session boundary. For example, a 5 m bar starting at 09:28 closes at 09:30 (truncated), and a new bar begins at 09:30 in the regular session.

BarState fields. Each rolling bar tracks: open, high, low, close, volume, trade_count, vwap (via cumulative numerator + denominator), bar_start (epoch ms), session tag.

Completed bar emission. When a new trade arrives whose timestamp falls outside the current bar window (or crosses a session boundary), the current bar is *completed* and returned to the Python caller. The caller (BarsBuilderService) is responsible for persisting to ClickHouse and pushing to WebSocket subscribers.

3.5.2 Implementation Phases

1. **Phase 1: Rust BarBuilder core**
   `rust/src/bars.rs` — `BarState`, `Timeframe` enum, `SessionCalendar`, `BarBuilder` as `#[pyclass]`. Methods: `ingest_trade()`, `get_current_bar()`, `flush()`. 8 timeframes, session-aware boundaries, VWAP tracking. Exposed via `lib.rs` in the existing `jerry_trader._rust` module.

2. **Phase 2: ClickHouse + Python service**
   ClickHouse schema for OHLCV bars (`ohlcv_bars` table, `ReplacingMergeTree`, partitioned by date). Python `BarsBuilderService`: orchestrates tick ingestion (reuses `UnifiedTickManager`) → Rust `BarBuilder` → ClickHouse writes → WebSocket broadcast of completed bars. ClickHouse replaces InfluxDB for bar storage (InfluxDB retained for snapshots/factors until full migration).

3. **Phase 3: Frontend integration**
   Update `chartDataStore.ts` to receive server-built bars via WebSocket (subscribe) and REST (historical query from ClickHouse). Remove frontend-side `updateFromTrade()` aggregation. TradingView Lightweight Charts `getBars()` / `subscribeBars()` backed by BarsBuilderService.

4. **Phase 4: Downstream consumers**
   FactorEngine consumes completed bars from BarBuilder instead of raw ticks for trade-rate / acceleration computation. Factor output migrated from InfluxDB to ClickHouse. Factor visualization overlay in the Chart module. Foundation for v3.0 stream bus: bar builder,

feature engine, strategy engine as parallel consumers off a shared tick bus.

3.5.3 10s Bootstrap — Historical Back-fill at Subscription Time

When a ticker is subscribed mid-session, the BarBuilder starts receiving WebSocket ticks from "now" forward, but has no historical bars for the earlier part of the day. The 10s bootstrap fills that gap by fetching trades from 04:00 ET to the current time and building bars retroactively.

Problem. A ticker added at, say, 11:04:13 ET via `_add_ticker()` immediately begins receiving WS ticks. But all 10 s bars from 04:00 ET through 11:04:10 ET are missing. These bars are needed by the Chart module and downstream consumers.

Trigger condition. On `_add_ticker()`, if `"10s"` is in the configured timeframes, the service queries ClickHouse for existing 10 s bars on the current date (`_has_10s_bars_today`). If the count is zero, a daemon thread is spawned for `_bootstrap_10s_bars(symbol)`.

Trade source dispatch (live vs. replay).

- **Live mode:** Polygon REST API via `fetch_polygon_trades(symbol)`. Returns trades from 04:00 ET to the current wall-clock time.
- **Replay mode:** Local Parquet files via the Rust function `load_trades_from_parquet(lake_da` `symbol, date, end_ts_ms)`. Tries the per-ticker partitioned file first (`trades_v1_partitioned/{ticker}/{date}.parquet`), falls back to the monolithic file (`trades_v1/{YYYY}/{MM}/{date}.parquet`) with a ticker filter. The replay clock's `clock_mod.now_ms()` is passed as `end_ts_ms` for predicate pushdown so only trades up to "now" are materialised.

Thread-safety: separate BarBuilder. The bootstrap runs on a daemon thread, while the WS consumer runs on the asyncio event loop. They must not share the same `BarBuilder` instance — if they did, a WS tick at 11:04 would jump the BarBuilder's internal state forward, causing all historical trades (04:00–11:04) to collapse into a single bar via `bar.update()`. **Fix:** The bootstrap creates a short-lived `BarBuilder(["10s"])` and feeds all historical trades through it using `ingest_trades_batch()`, producing an independent set of completed bars. The main BarBuilder continues to receive WS ticks unaffected.

Trade filtering at the WS boundary. The first WS tick timestamp for each ticker is recorded in `_first_ws_tick_ts[symbol]`. Historical trades

are filtered: only those with `ts < first_ws_ts` are fed to the bootstrap BarBuilder. This prevents double-counting in the overlap zone where both REST/Parquet and WS have data.

Meeting bar merge. The 10 s bar window that straddles the REST→WS boundary is the *meeting bar*. It is split between two sources:

Source	Trades in	Origin
REST prefix	[`bar_start` … `first_ws_ts`)	bootstrap BarBuilder
WS suffix	[`first_ws_ts` … `bar_end`)	main BarBuilder

The merge rules (`_merge_10s_bars`):

- `open` = REST (has earlier trades)
- `close` = WS (has later trades)
- `high` = max(REST, WS)
- `low` = min(REST, WS)
- `volume`, `trade_count` = sum
- `vwap` = volume-weighted average

ClickHouse uses `ReplacingMergeTree` with an `inserted_at` version column — if the WS meeting bar was flushed before the merge completes, the merged bar (inserted later) will survive deduplication.

State machine. Per-ticker state variables track the merge lifecycle:

1. `_first_ws_tick_ts[sym]` — set once on the very first WS tick.
2. `_meeting_bar_start[sym]` — the `bar_start` of the 10 s window containing the first WS tick. Detected via `get_current_bar(sym, "10s")` right after the first tick.
3. `_ws_meeting_bars[sym]` — the completed WS meeting bar, captured by `_capture_meeting_bar()` when the main BarBuilder emits it.
4. `_bootstrap_done` — set of tickers whose bootstrap has finished. Prevents spurious meeting-bar re-detection if WS ticks arrive after bootstrap completes (i.e. the first WS tick *is* in the same bar window that just got merged).

After the merge, items 2–3 are cleaned up (`.pop()`).

Performance. The Rust `ingest_trades_batch()` processes all trades in a single FFI call (pure Rust loop, no per-trade GIL interaction). Python dicts are built only for completed bars at the end. Benchmark: **4.2× faster** than per-trade `ingest_trade()` calls (39 ms vs. 164 ms for 441 K trades).

End-to-end flow.

```
_add_ticker("AAPL")
     |
```

```
+-- subscribe WS ticks (async)
|       |
|       v  _on_trade(): record first_ws_tick_ts,
|       |               detect meeting_bar_start
|
+-- _has_10s_bars_today("AAPL") -> False
        |
        v  Thread: _bootstrap_10s_bars("AAPL")
             1. load trades (REST or Parquet)
             2. filter ts < first_ws_ts
             3. BarBuilder(["10s"]).ingest_trades_batch()
             4. flush() -> get REST meeting bar
             5. separate clean bars vs meeting bar
             6. merge REST meeting bar + WS meeting bar
             7. append all to _pending_bars
             8. flush loop writes to ClickHouse
```

3.5.4 Multi-Timeframe Bootstrap — Extension Design Notes

The 10 s bootstrap currently only builds `"10s"` bars. Extending it to `1m`, `5m`, `15m`, `30m`, `1h`, `4h` would provide the same mid-session back-fill for every intraday timeframe. This section analyses what changes and what stays the same.

What stays the same.

- Trade source: exactly the same REST / Parquet fetch. Trades are timeframe-agnostic — one fetch feeds all timeframes.
- Thread-safety model: a separate `BarBuilder(timeframes)` instance.
- Trade filtering at `first_ws_ts`.
- The `_bootstrap_done` guard.
- ClickHouse persistence via `_pending_bars`.

What changes.

1. **Bootstrap BarBuilder timeframes.**
   Currently `BarBuilder(["10s"])`. Must become `BarBuilder(self.timeframes)` (or a filtered subset excluding `"1d"` and `"1w"` which have different bootstrap strategies).

2. **Meeting bar: per-timeframe, not per-ticker.**
   Currently there is one meeting bar per ticker (for 10 s). With N timeframes, each timeframe has its own meeting bar at the REST→WS boundary:

   - A 10 s meeting bar at, say, `11:04:10`

- A 1 m meeting bar at `11:04:00`
- A 5 m meeting bar at `11:00:00`
- A 15 m meeting bar at `11:00:00`
- …

The state variables must change from `Dict[str, int]` (ticker → bar_start) to `Dict[str, Dict[str, int]]` (ticker → timeframe → bar_start), and `_ws_meeting_bars` similarly.

3. **Meeting bar detection in `_on_trade()`.**
   Currently calls `get_current_bar(sym, "10s")` once. Must iterate over all bootstrap timeframes and record each meeting bar start.

4. **Meeting bar capture in `_capture_meeting_bar()`.**
   Currently hardcodes `timeframe == "10s"`. Must match any bootstrap timeframe and look up the correct entry in the per-timeframe dict.

5. **Merge function generalisation.**
   `_merge_10s_bars()` is already timeframe-agnostic in logic (OHLCV merge rules don't depend on bar duration). Only the hardcoded `"timeframe": "10s"` needs to become `rest_bar["timeframe"]`.

6. **ClickHouse existence check.**
   `_has_10s_bars_today()` is hardcoded for `timeframe = '10s'`. For multi-timeframe bootstrap it should check the *smallest* timeframe (10 s) — if 10 s bars exist, larger timeframes can be assumed present. Alternatively, check each timeframe independently, but this adds queries.

7. **Daily/weekly exclusion.**
   `1d` and `1w` bars should **not** be bootstrapped from intraday trades. The Chart module already loads these from Parquet history (local data loader). The bootstrap should filter to intraday timeframes only: `["10s","1m","5m","15m","30m","1h","4h"]`.

Estimated scope. The core fetch + filter + batch-ingest path is unchanged. The main work is refactoring the meeting-bar state machine from single-timeframe to per-timeframe dicts. The merge logic itself is already correct for any timeframe. Data volume is identical (same trades, different bar windows), so performance impact is negligible — `ingest_trades_batch` already processes all configured timeframes in the inner loop.

3.5.5 UTC/ET Timezone Conversion — DST-Aware Session Boundaries

Problem. Bar timestamps were displaying incorrectly — off by approximately 5 hours. For example, a bar starting at 04:00 AM ET (09:00 UTC)

was being stored and displayed as 04:00 UTC instead of 09:00 UTC.

The root cause was that the Rust `SessionCalendar` used a fixed UTC-5 offset (EST) to convert between UTC and Eastern Time. This fails during Daylight Saving Time when the offset should be UTC-4 (EDT). Since US markets observe DST from approximately March to November, bars were wrong for ∼8 months of the year.

Initial approach (rejected). Implementing DST calculation logic in Rust — computing the second Sunday in March and first Sunday in November, handling leap years, etc. This is complex, error-prone, and duplicates functionality already available in Python's `zoneinfo` module.

Adopted solution: Python handles timezone conversion. The architecture was simplified so that **Python converts all timestamps from UTC to ET before passing to Rust**, and Rust works *purely in ET time*.

1. **Data sources (Polygon/IB)** provide timestamps in UTC milliseconds.
2. **Python (`BarsBuilderService`)** converts UTC → ET using `zoneinfo.ZoneInfo("America/Ne` which correctly handles DST transitions.
3. **Rust (`BarBuilder`)** receives ET milliseconds and performs session-boundary calculations (04:00, 09:30, 16:00, 20:00 ET) using simple time-of-day arithmetic. No timezone conversion or DST logic in Rust.
4. **Python** converts bar timestamps from ET back to UTC before storing in ClickHouse and publishing to frontend.

Implementation details. Python helpers (bars_builder_service.py):

- `_utc_ms_to_et_ms(utc_ms: int) -> int`
  Converts UTC milliseconds to ET milliseconds. Accounts for DST by computing the timezone offset at the specific timestamp using `datetime.utcoffset()`.

```
utc_dt = datetime.fromtimestamp(utc_ms / 1000, tz=timezone.utc)
et_dt = utc_dt.astimezone(ZoneInfo("America/New_York"))
offset_seconds = et_dt.utcoffset().total_seconds()
return utc_ms + int(offset_seconds * 1000)
```

- `_et_ms_to_utc_ms(et_ms: int) -> int`
  Reverses the transformation. Interprets the ET millisecond value as a wall-clock time, localizes it to US/Eastern, then converts to UTC.
- `_convert_bar_et_to_utc(bar: dict) -> dict`
  Convenience wrapper that converts `bar["bar_start"]` and `bar["bar_end"]` from ET to UTC in-place.

Python call sites — all Rust interactions:

- `_on_trade()` — live WebSocket ticks: convert `ts_ms` (UTC) to ET before calling `bar_builder.ingest_trade()`.
- `_bootstrap_bars()` — historical trades: convert entire trade list from UTC to ET before calling `ingest_trades_batch()`.
- `_check_expired_loop()` — wall-time bar completion: convert `now_ms` (UTC) to ET before calling `check_expired()`.
- **After receiving bars from Rust:** all completed bars (from `ingest_trade`, `check_expired`, `flush`, etc.) are converted from ET back to UTC via `_convert_bar_et_to_utc` *before* storing in ClickHouse or publishing to Redis.

Rust changes (bars.rs):

- **Removed:** all DST calculation logic (EST/EDT offset constants, `is_dst()`, `nth_sunday_of_month()`, etc.).
- **Simplified:** `SessionCalendar` now assumes all input timestamps are in ET. Session boundaries (04:00, 09:30, 16:00, 20:00) are simple offsets from midnight.
- **Renamed parameters:** `ts_ms` → `et_ms` in function signatures for clarity.
- **Documentation:** added explicit contract stating that all timestamps must be in US/Eastern time, and that Python is responsible for time-zone conversion.

Data flow summary.

```
Polygon/IB (UTC)
  -> Python: UTC -> ET conversion (DST-aware via zoneinfo)
  -> Rust BarBuilder (works purely in ET)
  -> Python: ET -> UTC conversion
  -> ClickHouse / Frontend (UTC for storage/display)
```

Benefits.

- **DST handled correctly** by Python's battle-tested `zoneinfo`.
- **Rust code simplified** — no calendar arithmetic or DST edge cases.
- **Clear separation of concerns:** Python handles I/O and timezone complexity; Rust focuses on high-performance bar aggregation.
- **Standard practice:** storing data in UTC, converting to local time only for display or business-logic calculations.

4 Roadmap

-
- Break out Context build, model based.